Bangla Diarizz

Shamim Hasan
Department of Computer Science and Engineering
BNIST
Dhaka, Bangladesh
adilshamim696@gmail.com

M Nabil Shad Rahman
Faculty of Science and Technology
American International University-Bangladesh
Dhaka, Bangladesh
nyeowneko@gmail.com

Sadia Afrin Anika
Faculty of Science and Technology
American International University-Bangladesh
Dhaka, Bangladesh
anikasadia2723@gmail.com

Sojib Chandra Roy
Department of Computer Science and Engineering
Daffodil Institute of Information Technology, DIIT
Dhaka, Bangladesh
Rcsojib.cse1@gmail.com

***Abstract*—Automatic Speech Recognition (ASR) and Speaker Diarization have seen significant global progress in recent years. However, their practicality relative to Bangla - the world's seventh most spoken language – has not been worked upon much, due to a scarcity of long-form, multi-speaker datasets. This paper presents two slightly improved models that jointly performs Bangla transcription and speaker diarization made as part of the DL Sprint 4.0 competition. The transcription model leverages [mention your core tech, e.g., Transformer-based encoders or fine-tuned Whisper/Wav2Vec2]. On the other hand, Bangla Diarizz, the diarization model, offers an improved pipeline of the Pyannote Community 1 pipeline by swapping models, components and further fine-tuning. Evaluation on the competition dataset confirmed the effectiveness of our work, dropping the original DER from 41% to about 19.264%.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Despite being spoken by over 250 million people, Bangla remains a significantly low-resource language in the context of Automatic Speech Recognition (ASR). The difference stands out: while English benefits from over 10,000 hours of labeled datasets, Bangla resources are limited to approximately 1,500 hours. This data scarcity, compounded by complex phonology, broad dialectal diversity, and frequent environmental noise, creates a substantial barrier to developing robust systems for real-world applications [1], [2]. Furthermore, there is an even higher lack of labelled data for diarization with only about 100 hours worth of easily available data.

Current state-of-the-art architectures, such as Wav2Vec 2.0 [3] and Whisper [4], show remarkable performance in high-resource multilingual contexts but falter when applied to Bangla, despite being trained on it. Bangla, having a wide wariety of dialects, on top of having low resources, In noisy environments, Whisper's Word Error Rate (WER) has been observed to exceed 30% [5], demonstrating a critical inability to manage the language's phonetic intricacies and external acoustic interference. Pyannote's Community 1 model shows a DER of about 41% in the test data for the competition.

To address this, Bangla Diarizz, further modifies the Pyannote Community 1 by changing key components along with further post processing adapts the model to better adapt to speaker variabilitiy and identification, dropping to 19.264% on the test data.

II. Related Work

A. Classical and Modular Diarization Approaches

The standard speaker diarization pipeline has undergone notable evolution over three decades. Early systems relied on Gaussian Mixture Models (GMMs) and Bayesian Information Criterion (BIC) for speaker change detection, popularized through the NIST Rich Transcription evaluations of the early 2000s [3]. The introduction of i-vectors and, later, x-vectors as speaker embeddings transformed clustering quality by providing discriminatively trained, fixed-dimensional representations of variable-duration speech segments. These developments are surveyed extensively in Park et al. [2] and Anguera et al. [11], whose taxonomies organize techniques into front-end processing, segmentation, embedding, and back-end clustering.

B. Neural End-to-End Approaches

End-to-End Neural Diarization (EEND) represents a paradigm shift in which the entire diarization task is formulated as a sequence-to-sequence prediction problem optimized with permutation-invariant losses [7]. SA-EEND and EEND-EDA extend this framework to handle an unknown number of speakers, while PixIT combines speech separation with diarization. Despite their conceptual elegance, EEND systems are data-hungry and typically underperform modular systems in low-data regimes — a consideration especially relevant to the Bengali setting. The TalTech-IRIT-LIS submission to DISPLACE 2024 [6], which achieved a DER of 27.1% through an EEND ensemble, illustrates the data requirements needed to reach such performance levels.

C. Bengali Speech Technology

Bengali long-form speech processing has historically been underrepresented in the research literature. The DISPLACE challenge series [5] touched on Bengali within its multilingual Indian-language track but did not provide Bengali-specific diarization benchmarks. The most comprehensive resource to

date is the Bengali-Loop benchmark introduced by Tabib et al. [1], which establishes a 14-recording test set with fully manual annotations and reports a pyannote.audio baseline DER of 40.08%. Prior Bengali ASR work has examined crowdsourced read-speech datasets such as OpenSLR and Common Voice contributions, but none specifically addresses the multi-speaker long-form diarization setting. Our work builds directly upon the Bengali-Loop dataset and evaluation protocol.

D. The Pyannote Ecosystem

Pyannote.audio [8] provides a modular, PyTorch-based pipeline for speaker diarization centred on a learnable segmentation model and pluggable embedding backends. The Segmentation 3.0 model uses a powerset multi-class training objective that jointly models speech activity and local speaker identity, while clustering is performed via VBx or agglomerative hierarchical clustering (AHC). The community-released Pyannote-Community-1 pipeline maintains this structure while relaxing certain access constraints, making it the natural starting point for our adaptation work.

III. System Architecture and Methodology

A. The Pipeline

The Bangla Diarizz system is constructed by modifying the Pyannote-Community-1 pipeline at twokey injection points: the segmentation model and the speaker embedding model. The base pipeline provides a well-optimized data flow, incorporating sliding-window frame extraction, internal audio validation, VBx clustering, and the overall coordination logic. Our modifications are surgical and non-invasive to this data flow, replacing only the neural model components while preserving all surrounding infrastructure, as it offers optimized functions.

The full inference pipeline follows this sequence:

1. Audio Preprocessing: WAV to mono, 16 kHz (torchaudio), loading the audio beforehand in order to increase processing efficiency.
2. Speaker Segmentation: gomiiie's Bangla fine-tuned Segmentation; per-frame posteriors (powerset output).
3. This replaces the pyannote's original segmentation 3.0 model and was fine-tuned on the public competition data provided.
4. Feature Aggregation: Sliding-window windowing, step = 0.1 (90% overlap).
5. Speaker Embedding Extraction: WeSpeaker ResNet34-LM; 256-dimensional embeddings per segment, replaces the pyannote's embedding model.
6. VBx Clustering: Variational Bayes x-vector clustering, cosine threshold = 0.45 with PLDA.
7. Post-processing: Chronological relabelling + gap stitching (gap <= 2s).

B. gomiiie's Bangla-finetuned-segmentation

The Pyannote Segmentation 3.0 model employs a powerset multi-class training objective that jointly predicts local speech activity and local speaker identity within short windows. The model is built on a SincNet front-end, followed by bidirectional LSTM layers and a final classification head. Its compact size of approximately 1.47 million parameters makes it well-suited for fine-tuning on limited in-domain data.

We fine-tune the pretrained Segmentation 3.0 checkpoint on the 10-file competition training set using the diarizers library, which provides a Hugging Face-compatible training loop specifically designed for pyannote-format segmentation models. Fine-tuning enables the model to adapt its frame-level classification boundaries to the acoustic characteristics of Bengali speech: shorter speaker turns, distinct prosodic patterns, and domain-specific background conditions present in Bangla drama recordings.

A critical tuning decision is the min_duration_off parameter, set to 0.1 s. This instructs the VAD stage to treat pauses shorter than 100 ms as continuing speech rather than silence, preventing over-segmentation caused by brief intra-utterance pauses — an especially important consideration for naturally paced Bengali conversational speech.

C. Speaker Embedding Model: WeSpeaker ResNet34-LM

The default pyannote.audio embedding model is replaced with WeSpeaker ResNet34-LM [9], a ResNet-based speaker verification model with approximately 6.63 million parameters trained on the VoxCeleb corpus using the WeSpeaker framework. ResNet34 architectures have demonstrated competitive performance on speaker verification benchmarks, and the large-margin (LM) training objective produces embeddings with better intra-speaker compactness and inter-speaker separation relative to softmax-trained counterparts. The combined effective parameter count of the two neural components is approximately 8.1 million - an intentionally lightweight configuration designed to balance accuracy with inference efficiency, given that the competition scoring allocates 30% of the online score to the Real-Time Factor (RTF).

D. Clustering Hyperparameters

Clustering is performed by VBx as implemented within the pyannote pipeline. The pyannote already offers built in clustering algorithms, from which VBx was chosen, compared to the original AHC. speaker merging: a lower threshold produces more conservative merges (higher speaker count), while a higher threshold aggressively collapses speakers. Following ablation experiments on the training split, we fix the clustering threshold at 0.45. Table 1 summarizes all active hyperparameters.

TABLE I

Overall Hyperparameters	
Hyperparameter	***Value***
Min_stitch	2.0s
Min_duration_off	0.1s
Segmentation_step	0.1

Clustering_threshold	0.45
Min_speakers	2
Max_speakers	20

Fig. 1. Overall hyperparameters

D. Post-Processing: Stitching and Relabelling

Raw pipeline output is processed by the get_chronological_segments() function, which performs two operations. First, speakers are relabelled in strict chronological order of first appearance as SPEAKER1, SPEAKER2, …, SPEAKERN. This is semantically neutral with respect to DER evaluation (since the evaluator performs optimal label permutation matching) but produces a cleaner, human-readable output. Second, successive segments attributed to the same speaker with a temporal gap of at most min_stitch = 2 s are welded into a single segment. This stitching step addresses a well-known failure mode in sliding-window diarization pipelines where brief acoustic interruptions or model uncertainty produce artificial fragmentation of a single speaker turn.

IV. Evaluation

A. Evaluation Protocool

Competition submissions are evaluated using the Diarization Error Rate (DER), which accumulates three types of errors: False Alarm (FA), predicted speech in regions of silence; Missed Speech (MISS), silence predicted over true speech; and Speaker Confusion (CONF), correct speech detection but incorrect speaker assignment. The overall competition score combines DER performance (70% weight) with inference speed measured via Real-Time Factor (30% weight), using percentile ranking across all participants.

DER = (FA + MISS + CONF) / Total Reference Speech Duration.

The competition score is then Score = max(0, 100 × (1 - DER)). Speaker IDs in predicted output are treated as opaque labels; the evaluator automatically computes the optimal bijection between predicted and reference speaker labels to minimize errors.

B. Baseline Comparisons

TABLE II

Benchmarks	
System	***DER(%)***
Pyannote Diarization 3.0	31.5%
Pyannote Community-1	29.895%
Bangla Diarizz	19.246%

Fig. 2. Benchmarks

Furthermore, during fine-tune training, the segmentation model displayed a DER of ~9.5% compared to starting off with ~22% at the very first step with the pyannote segmentation model.

C. Inference Time Analysis

The primary source of speed-up was the introduction of a dedicated audio preprocessing function (load_audio_for_pyannote) that delivers audio to the pipeline in a pre-validated tensor format. This eliminated repeated internal format inspections and on-the-fly resampling within the pipeline's audio loading subsystem.

TABLE III

Inference Time		
Implementation Version	***Total Inference Time***	***% change***
Bangla Diarizz v1	~1 hr 22 min	Baseline
Bangla Diarizz v8	~35 min 56s	134%

Fig. 3. Benchmarks

D. Clustering Hyperparameter Sensitivity

The clustering threshold and stitching gap represent the two parameters with the greatest empirical impact on DER. A threshold below 0.40 leads to over-segmentation (excessive speaker counts), while values above 0.55 merge genuinely distinct speakers — the latter being especially damaging to the speaker confusion component of DER. The stitching gap of 2 s was selected empirically: intervals shorter than 2 s between same-speaker segments in Bangla drama commonly correspond to brief pauses, breath, or laughter rather than a change of speaker turn. The segmentation step of 0.1 (i.e., 10% non-overlapping stride, equivalent to 90% overlap) provides dense frame-level predictions at the cost of increased computation. In our T4 GPU environment this trade-off is acceptable; for CPU inference or more aggressive RTF optimization, a larger step (e.g., 0.25) would reduce computation by approximately 60% with a modest increase in segmentation boundary imprecision.

V. Novelty

The principal novelties of Bangla Diarizz relative to prior published work are:

1. Bengali-specific segmentation fine-tuning: To our knowledge, this is the first publicly documented fine-tuning of the pyannote Segmentation 3.0 model specifically on a Bengali diarization corpus. Prior Bengali diarization work has either used off-the-shelf models without adaptation.
2. Surgical pipeline modification: Rather than retraining the entire pipeline end-to-end, we exploit the modular architecture of pyannote.audio to replace precisely the two components most sensitive to language-specific acoustics (segmentation and embedding), leaving clustering logic intact. This strategy is both data-efficient and computationally tractable.

VI. Future

The Bengali-Loop baseline results [1] establish that the performance gap between even lightweight systems and the unmodified pyannote.audio model is substantial: 33 percentage points separate the weakest and strongest language-agnostic baselines. Our hypothesis — that domain-adapted segmentation alone can close a significant fraction of the remaining gap above the pyannote.audio baseline — is grounded in prior work demonstrating that segmentation errors account for a disproportionate share of DER in multi-speaker long-form audio.

Post-processing analysis of the segmentation model output reveals that the primary error mode before stitching is over-fragmentation: single speaker turns are split into multiple sub-segments by brief VAD-off events of 100–800 ms. The stitching post-processing step significantly reduces this fragmentation for same-speaker successive segments, at the acceptable risk of sometimes merging turns across a brief interruption by another speaker.

The WeSpeaker ResNet34-LM model, despite being trained on VoxCeleb (predominantly English celebrity speech), produces embeddings that cluster reasonably well for Bengali speakers in a VBx framework. This is consistent with evidence from cross-lingual speaker verification literature that deep speaker embeddings capture phonation and vocal tract characteristics that generalize across languages. However, the clustering threshold of 0.45 is lower than thresholds commonly reported for English, suggesting that Bengali vocal features yield slightly less well-separated embedding clusters - an expected consequence of domain mismatch.

Failure Modes and Limitations Several systematic failure modes warrant discussion. First, the system does not handle overlapping speech: pyannote's VAD assigns each frame to a single speaker, so concurrent speech is attributed to whichever speaker's signal is dominant. Second, extremely short speaker turns (< 0.5 s) are often absorbed into neighbouring segments by the stitching step, inflating confusion errors for recordings with rapid turn-taking. Third, recordings with more than five speakers introduce clustering instability at the 0.45 threshold; permitting max_speakers=20 alleviates this at the cost of occasional over-segmentation.

Several avenues emerge to further enhance Bengali speech systems. From a modeling perspective, transitioning to overlap-aware diarization via the pyannote powerset framework would address the failure modes in overlapping speech, while multi-condition fine-tuning (noise, reverb) would improve acoustic robustness. Regarding speaker embeddings, training an in-language model on a Bengali-specific corpus is essential; the observed clustering threshold of 0.45 suggests that VoxCeleb-trained embeddings do not adequately capture Bengali vocal characteristics.

From a data standpoint, expanding the Bengali-Loop corpus to include spontaneous dialogue and regional dialects (e.g., Sylheti, Chittagong) would enable data-intensive EEND-based approaches. Finally, implementing a DiarizationLM-style post-processing stage—leveraging ASR transcripts to resolve speaker ambiguity through semantic context—offers a promising path toward achieving human-level performance on the Bengali-Loop benchmark.

VII. Conclusion

This paper has presented Bangla Diarizz, a domain-adapted speaker diarization system for Bengali long-form audio. By surgically replacing the segmentation and embedding modules of the Bangla Diarizz | Team Envisage | DL Sprint 4.0, BUET CSE Fest 2026 | Page 10 Pyannote-Community-1 pipeline with a Bengali fine-tuned Segmentation 3.0 model and WeSpeaker ResNet34-LM embeddings respectively, we construct a system that is both acoustically adapted to Bengali speech and inference-efficient. A targeted post-processing stage further reduces fragmentation artefacts prevalent in sliding-window diarization. The combined approach achieves a ~56% reduction in wall-clock inference time relative to a naive implementation and is positioned to improve upon the 40.08% DER baseline established for the Bengali-Loop benchmark [1]. Our work demonstrates that the modular architecture of modern diarization pipelines, when combined with fine-tuning on even limited in-domain data, offers an accessible and effective pathway to language-specific diarization for under-resourced languages. We hope that Bangla Diarizz, its public inference notebook, and the associated Bengali-Loop benchmark collectively lower the barrier for future Bengali speech technology research.

References

[1] Sarker, M. Hasan, and M. Ali, "Challenges in developing bangla asrfor real-world deployment," in Proceedings of the Language Resources and Evaluation Conference (LREC), pp. 991–997, ELRA and ICCL, 2024.

[2] M. Rahman and M. Hossain, "Environmental noise and dialectal variation in bangla asr: An empirical study," in IEEE Automatic Speech Recognition and Understanding Workshop (ASRU), pp. 189–194, IEEE, 2022.

[3] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[4] A. Radford, J. W. Kim, T. Xu, et al., "Robust speech recognition via large-scale weak supervision," in ICML, 2023.

[5] M. Islam, T. Hossain, and S. Rahman, "Evaluating whisper on south asian languages: The case of noisy bangla," in IEEE Spoken Language Technology Workshop (SLT), pp. 201–206, 2024.

[6] H.M. Shadman Tabib et al., "Bengali-Loop: Community Benchmarks for Long-Form Bangla ASR and Speaker Diarization," arXiv:2602.14291, 2026. https://doi.org/10.48550/arXiv.2602.14291

[7] D. Park, K. Han et al., "A Review of Speaker Diarization: Recent Advances with Deep Learning," Computer Speech and Language, vol. 72, p. 101317, Elsevier, 2022.

[8] M.J. Picheny, A. Narayanan et al., "Speaker Diarization: A Review of Objectives and Methods," Applied Sciences, vol. 15, no. 4, MDPI, 2025.

[9] S. Wang et al., "DiarizationLM: Speaker Diarization Post-Processing with Large Language Models," arXiv:2401.03506, 2024.

[10] P. Kodali et al., "The Second DISPLACE Challenge: DIarization of SPeaker and LAnguage in Conversational Environments," arXiv:2406.09494, 2024.

[11] J. Villalba et al., "TalTech-IRIT-LIS Speaker and Language Diarization Systems for DISPLACE 2024," arXiv:2407.12743, 2024.

[12] Y. Fujita, N. Kanda et al., "End-to-End Neural Speaker Diarization with Self-Attention," in Proc. IEEE ASRU, pp.296–303, 2019.

[13] H. Bredin et al., "pyannote.audio: Neural Building Blocks for Speaker Diarization," in Proc. IEEE ICASSP, Barcelona, Spain, pp. 7124–7128, 2020.

[14] S. Wang et al., "WeSpeaker: A Research and Production Oriented Speaker Embedding Learning Toolkit," in Proc.IEEE ICASSP, pp. 1–5, 2023. Available: https://github.com/wenet-e2e/wespeaker

[15] A.M. Amimul Ehsan et al., "DL Sprint 4.0 — Bengali Speaker Diarization," Kaggle Competition, BUET CSE Fest 2026. https://kaggle.com/competitions/dl-sprint-4-0-bengali-speaker-diarization-challenge

[16] X. Anguera Miro et al., "Speaker Diarization: A Review of RecentResearch," IEEE Transactions on Audio, Speech, and Language Processing, vol. 20, no. 2, pp. 356–370, 2012.